NAME : RUSHI A. PATEL

EN. NO : 2302030400116

ROLL NO : 48

CLASS : 4_CE_B

Pandas: Unleashing the Power of Data Analysis in Python

Pandas - Your Data Powerhouse in Python

Introduction - Why Pandas?

Data Analysis Importance

Data analysis plays a crucial role in decision-making today.

Pandas Introduction

Pandas is a powerful library designed for efficient data analysis in Python.

Challenges with Raw Data

Raw data often comes in various formats and requires extensive cleaning.

Key Strengths

Includes data structures like Series and DataFrame, plus integration with libraries.

Core Data Structures: Series

1 **Pandas Series Definition**

A one-dimensional labeled array capable of holding any data type.

2 **Example Structure**

Illustrated with Python code for creating a Series.

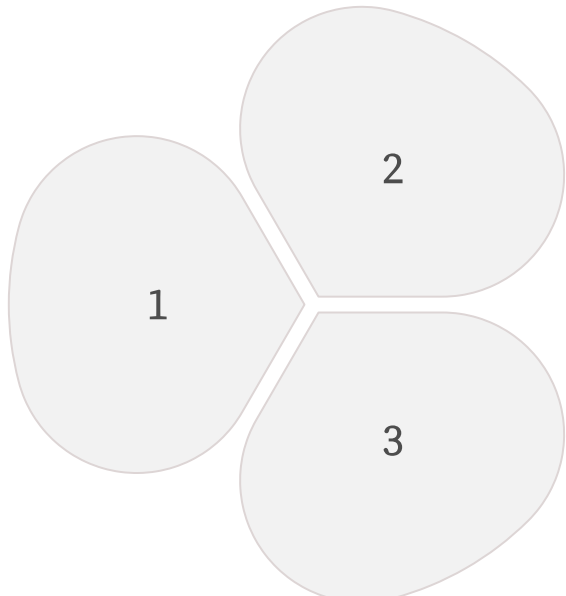
3 **Accessing Elements**

Demonstrating how to access elements by label and index.

Core Data Structures: DataFrame

What is a DataFrame

A two-dimensional labeled data structure with columns of different types, akin to a table or spreadsheet.



Creating a DataFrame

Using a dictionary to create a DataFrame for structured data.

Accessing Data

Techniques to access columns and rows in a DataFrame for data manipulation.

Reading and Writing Data

Pandas File Reading

Pandas can read data from various file formats.

Reading CSV Example

Read CSV file using pd.read_csv() function.

Reading Excel Example

Read Excel file using pd.read_excel() function.

Data Writing Methods

Use to_csv() and to_excel() methods to write data.

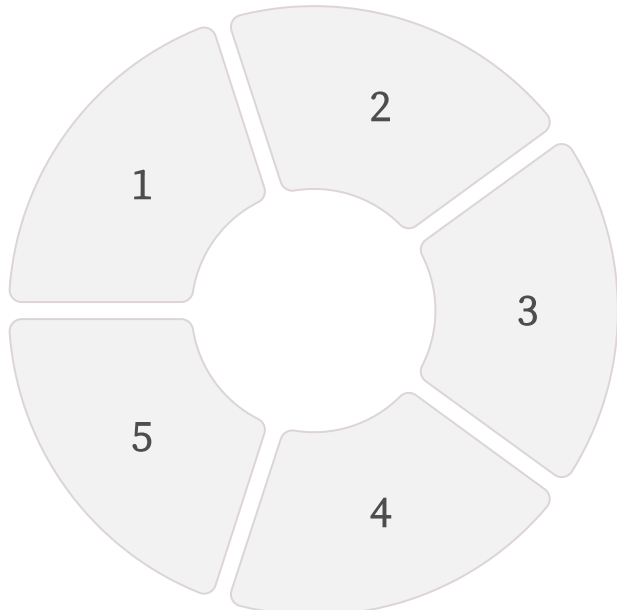
Data Inspection Techniques

.head()

Display the first few rows.

.shape

Get Dimensions of the DataFrame.



.tail()

Display the last few rows.

.info()

Get a concise summary of the DataFrame.

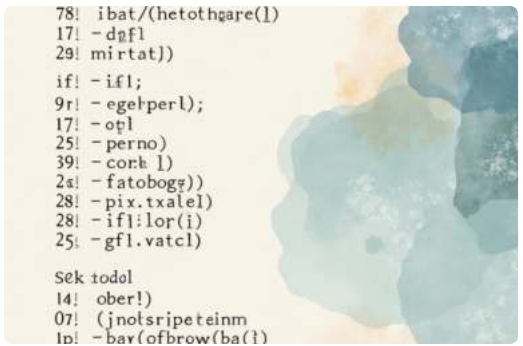
.describe()

Generate descriptive statistics of numerical columns.

Data Selection and Filtering

Column Selection

e.g., df['Name']



Row Selection

Using .loc (label-based) and .iloc (integer-based)

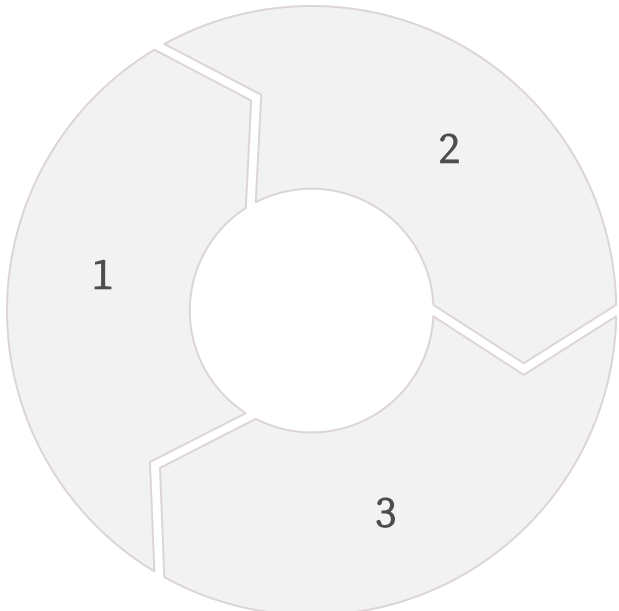
Conditional Filtering

e.g., df[df['Age'] > 27]

Data Cleaning - Handling Missing Values

Importance of Missing Data

Understanding how missing values can affect analysis.



Identifying Missing Values

Use .isnull() to check for any missing values.

Handling Missing Values

Methods like .fillna() and .dropna() for managing data gaps.

Data Manipulation - Adding, Modifying, and Deleting

Adding New Columns

Example: df['Salary'] = [60000, 75000, 65000]

Modifying Existing Values

Change values in existing columns as needed.

Deleting Columns

Use del df['ColumnName'] or df.drop('ColumnName', axis=1).

Data Aggregation and Grouping

Grouping Data

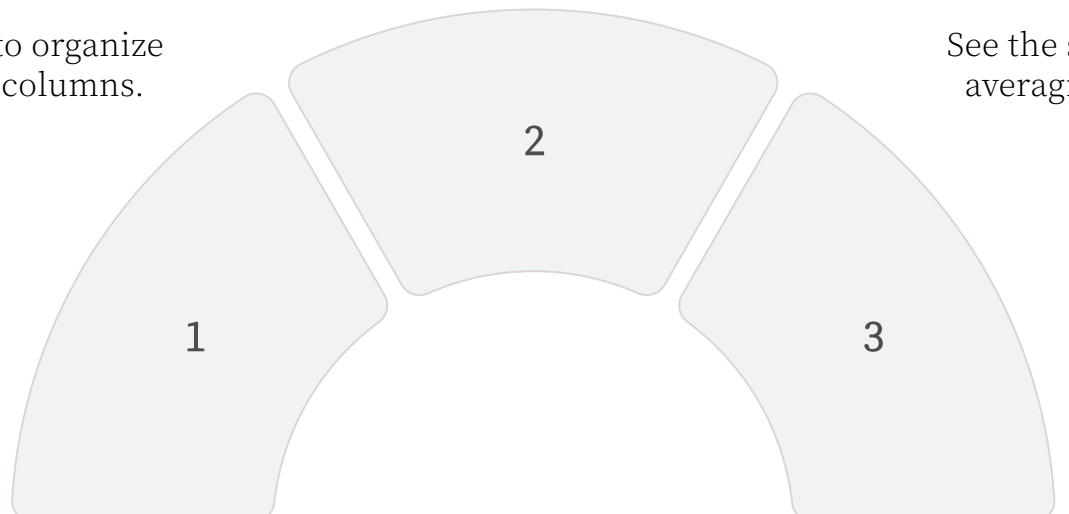
Utilize .groupby() to organize data by specified columns.

Aggregation Functions

Apply functions like .mean(), .sum(), .count() on grouped data.

Example Code

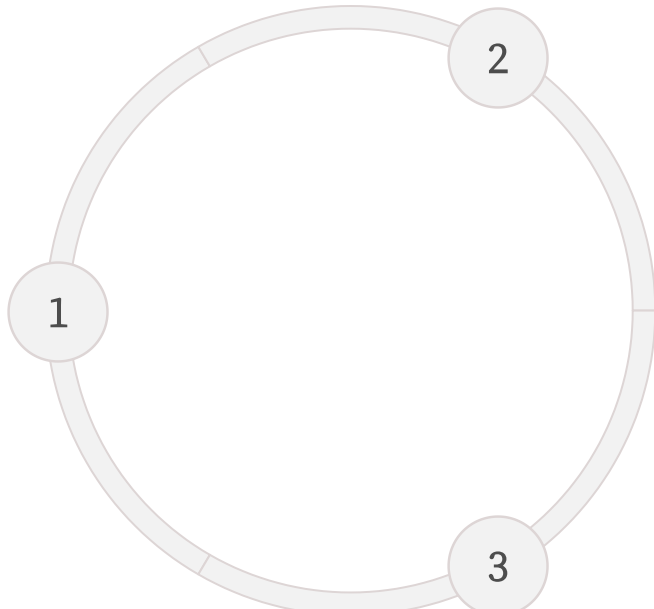
See the sample code for averaging age by city.



Merging and Joining DataFrames

Combining DataFrames

Utilize .merge() and .join() methods effectively.



Types of Joins

Includes inner, outer, left, and right joins.

Use Cases

Understanding which join to use based on data requirements.

Conclusion and Further Exploration

Key Benefits

Pandas offers powerful data manipulation and analysis tools.

Wide Applications

Widely used in data science, machine learning, finance, and more.

Further Exploration

Explore the comprehensive Pandas documentation and resources.

GitHub Mini Project Link

https://github.com/rap892005/pp-2302030400116

Mini Project: Analyzing a Simple Dataset with Pandas

This mini project showcases the use of Pandas to analyze a dataset consisting of student scores in different subjects. We will create a CSV file, write a Python script to load the dataset, and perform various analyses to derive insights such as average scores.

1

Create a CSV file

student_scores.csv with student scores data.

2

Write Python Script

analyze_scores.py to load and analyze the dataset using Pandas.

3

Perform Analysis

Calculate average scores and identify top-performing students.